

MoA Compiler Architecture: From Dissertation ψ and Ω to Verified Portable Kernels

Lenore Mullin — University at Albany, SUNY — Dissertation 1988 + Papers I-V (2026)

September 30, 2026

Abstract

Compiler spec from hand-derived Mathematics of Arrays in Mullin 1988 dissertation (123 pages, ψ , Ω , take $\uparrow$, drop $\downarrow$, catenation, rotate θ , reverse ϕ , transpose $\odot$, compress f , expand $\backslash$) to Papers I-V: arXiv:2606.07713, HAL:05659212, arXiv:2607.19456, HAL:05734881, arXiv:2609.33916. IR is Denotational Normal Form (DNF) achieving $O(nd_k + nd_v)$ minimal vs $O(n^2)$, verified $\|\text{err}\|_\infty \leq 2e-7$ and $= 0$ exact IEEE-754. Backend is Operational Normal Form (ONF) via $\gamma(\vec{v}, \vec{s}) = \sum v_k \prod s_j$ with deterministic portable paradigms only: OpenMP, OpenACC, OpenMPI, and deterministic flags fitting predictive model $T \approx \alpha F + \beta M$, $\beta \gg \alpha$ (100-1000x DRAM vs FLOP).

1 Dissertation Core: Full ψ as Derived by Hand

From pages 20-50: Original definition 2.0 $\vec{i}\psi\xi$ only for full index. Extended (your extension) for partial $\vec{i}$ where $\tau\vec{i} < \delta\xi$: $\rho(\vec{i}\psi\xi) = \tau\vec{i} \downarrow \rho\xi$ giving subarray sections. This is take/drop foundation.

Lemma L1-L5 proofs in dissertation show:

$$(\xi_l, \xi_r)\psi\xi \equiv (\xi_l\psi\xi), (\xi_r\psi\xi) \quad (L4)$$

$$\xi_l\psi(\sigma \text{ op } \xi_r) \equiv \sigma \text{ op } (\xi_l\psi\xi_r) \quad (L5)$$

i.e., ψ distributes over catenation and unary ops — core of psi-reduction to DNF.

Take $\vec{v} \uparrow \xi$ (Def 1.0): $\rho(\vec{v} \uparrow \xi) \equiv (\text{abs } \vec{v}), (r\vec{v}) \downarrow \rho\xi$, $\vec{i}\psi(\vec{v} \uparrow \xi) \equiv (\vec{i} + \vec{j})\psi\xi$ — selects first $|\vec{v}|$ elements. Drop $\vec{v} \downarrow \xi$ (Def 1.1): complement, $(r\vec{v} \uparrow \rho\xi) - \text{abs } \vec{v}$ — crucial for KV-cache $K_{old}\#k_t$ as take+drop+cat. Rotate θ , reverse ϕ (Def 0.0-2.0): $\rho\phi\xi \equiv \rho\xi$, $< i > \psi\phi\xi \equiv < (\rho\xi)[0] - (i+1) > \psi\xi$. Transpose $\vec{v} \odot \xi$ (Def 3.1): $\rho(\vec{v} \odot \xi) \equiv (\rho\xi)[gu \vec{v}]$, $< i > \psi(\vec{v} \odot \xi) \equiv \vec{i}[gu \vec{v}]\psi\xi$ — but DNF eliminates it algebraically, no buffer.

This full ψ calculus is the frontend parser: PyTorch op $\rightarrow$ take/drop/ ψ expression.

2 Omega Subset: What Compiler Needs

Dissertation Chap IV-VI defines Ω lifting $f_{\Omega(\sigma_l, \sigma_r)}$. Full Ω is powerful but compiler needs only 4 deterministic patterns covering Papers I-V:

1. $+_{\Omega\langle 0,1 \rangle}, \times_{\Omega\langle 0,1 \rangle}$: broadcast add/mul for $QK^T/\sqrt{d_k}$ + mask and softmax numerator.
2. $+_{\Omega\langle 1,1 \rangle}$: reduction $\sum_{d_k}$ for dot product, $\rho = \sigma_l \downarrow \rho\xi_l \# \sigma_r \downarrow \rho\xi_r$ with $m = 1$.
3. $\max_{\Omega\langle 1 \rangle}$: stable softmax max, plus $+_{\Omega\langle 0,1 \rangle}$ for subtract.
4. Selection ψ for GQA/MQA: h_q/h_{kv} proven reduction via $\vec{i}\psi$ modulo.

No arbitrary Ω nesting — only these 4 give DNFs in Papers I-III with $\|\text{err}\|_\infty = 0$.

3 Portable Paradigms Only: OpenMP, OpenACC, OpenMPI

Constraint: NO CUDA/HIP. Only standards.

OpenMP (CPU): `#pragma omp parallel for schedule(static) proc_bind(close)` over ONF. γ^{row} maps to offset. Explains Paper V 535x NUMA penalty vs $< 3x$ oversubscription — same DNF, different *OMP_PLACES*.

OpenACC (GPU): `#pragma acc parallel loop gang worker vector_length(128) independent`. Paper III verified exact. Atomic contention fix 2.00x $\rightarrow$ 2.5x via `#pragma acc atomic` restructuring in ONF only, DNF fixed.

OpenMPI (Distributed): $\rho A = \langle n \rangle = \langle n_0 \# n_1 \rangle$. $K_{new} = K_{old} \# k_t$ is *MPI_Scatterv/Gatherv* with ψ -selection halo. KV-cache $O(d_k + d_v)$ append proven by cat lemma.

C vs Fortran anomaly: C+OpenMP faster CPU, Fortran+OpenACC faster GPU, 3.17x time matches 3.35x stall — compiler picks γ^{row} vs $\gamma^{col}(\vec{v}, \vec{s}) = \sum v_k \prod_{j=0}^{k-1} s_j$.

4 Deterministic Flags and Pragmas for Predictive Model

Predictive model requires reproducible γ costs. Your prior work (Mullin 2023 GPU energy, Mullin & Hains 2025 parallel generation, Thomas 2021 DGEMM) uses $T \approx \alpha F + \beta M$.

Allowed deterministic:

- `-O2 -std=c11 -fopenmp -fno-fast-math -fno-unsafe-math-optimizations -DOMP_DETERMINISTIC`
- `-acc -Minfo=accel -ta=tesla:cc80 -fopenacc deterministic lowering`
- `-DOMPI_DETERMINISTIC` fixed reduction order

Forbidden: `-O3 -Ofast -ffast-math -funroll-loops` — breaks IEEE exactness and invalidates model.

Deterministic pragmas map 1-1 to ONF:

```
#pragma omp simd aligned(q,k,v:64) // gamma coalesced
#pragma acc loop seq // scalar G_A,G_S in backward
```

Measured vs predicted: 2-100x speedup, 2-50x energy from Paper I.

5 Compiler Passes — Full Pipeline

```
def compile(pytorch_src, target='Anvil'):
    moa_ast = parse_take_drop_psi(pytorch_src) # Diss. Def 1.0,1.1,L1-L5
    omega_ast = lift_to_0omega_subset(moa_ast) # 4 patterns only
    dnf = psi_reduce_full(omega_ast) # full calculus
    verify(dnf, pytorch_src, eps=0) # Paper III exact
    onf = gamma_lower(dnf, layout='row', deterministic=True)
    onf = optimize_onf_portable(onf, target, paradigms=['OMP','ACC','MPI'])
    # fixes: 2.00x atomics  $\rightarrow$  2.5x, 535x NUMA, 3.17x stall
    emit(onf, flags='-O2 -fno-fast-math -fopenmp -fopenacc')
```

Paper I-V kernels all compile through this: forward, backward G_V, G_Q, G_K eliminating G_A, G_S, G_{sc} , fused $O(n^2)$ avoidance, decode $(d_k + nd_k + nd_v + d_v) * 4B$, GQA/MQA h_q/h_{kv} , block RMSNorm+gated MLP.

Next: upload predictive model papers to annotate DNF nodes with cost βM for exascale.